```
import random
import time

# Bubble Sort
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]


# Selection Sort
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]


# Insertion Sort
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key


# Merge Sort
def merge(arr, left, mid, right):
    L = arr[left:mid + 1]
    R = arr[mid + 1:right + 1]

    i = j = 0
    k = left

    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1

    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1

    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1


def merge_sort(arr, left, right):
    if left < right:
        mid = (left + right) // 2
        merge_sort(arr, left, mid)
        merge_sort(arr, mid + 1, right)
        merge(arr, left, mid, right)


# Quick Sort
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
```

```
    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1


def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)


# Main Program
n = 100
arr = [random.randint(0, 999) for _ in range(n)]

print("Number of Elements =", n)
print()

# Bubble Sort
temp = arr.copy()
start = time.perf_counter()
bubble_sort(temp)
end = time.perf_counter()
print(f"Bubble Sort Time    : {(end - start) * 1_000_000:.2f} microseconds")

# Selection Sort
temp = arr.copy()
start = time.perf_counter()
selection_sort(temp)
end = time.perf_counter()
print(f"Selection Sort Time : {(end - start) * 1_000_000:.2f} microseconds")

# Insertion Sort
temp = arr.copy()
start = time.perf_counter()
insertion_sort(temp)
end = time.perf_counter()
print(f"Insertion Sort Time : {(end - start) * 1_000_000:.2f} microseconds")

# Merge Sort
temp = arr.copy()
start = time.perf_counter()
merge_sort(temp, 0, n - 1)
end = time.perf_counter()
print(f"Merge Sort Time     : {(end - start) * 1_000_000:.2f} microseconds")

# Quick Sort
temp = arr.copy()
start = time.perf_counter()
quick_sort(temp, 0, n - 1)
end = time.perf_counter()
print(f"Quick Sort Time     : {(end - start) * 1_000_000:.2f} microseconds")
```

```
Number of Elements = 100

Bubble Sort Time    : 442.09 microseconds
Selection Sort Time : 250.55 microseconds
Insertion Sort Time : 238.93 microseconds
Merge Sort Time     : 205.18 microseconds
Quick Sort Time     : 127.19 microseconds
```